



3.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**PSEUDOCODE WRITING**
← Back

**QUESTIONS**

**Q1** Student Management System (BL3 - Apply) 10 marks

**Q2** Library Book Management (BL3 - Apply) 10 marks

**Q3** Employee Salary Calculation (BL3 - Apply) 10 marks

**Q4** Shape Area Calculator using Polymorphism (BL3 - Apply) 10 marks

**Q5** Vehicle Registration System (BL3 - Apply) 10 marks

**Q6** Bank Account Hierarchy (BL4 - Analyze) 10 marks

**Q7** Hospital Patient Record System (BL4 - Analyze) 10 marks

**Q8** E-commerce Product Catalog (BL4 - Analyze) 10 marks

**Q9** University Staff Hierarchy (BL4 - Analyze) 10 marks

**Q10**

**Q3 Employee Salary Calculation [BL3 - Apply]** 10 marks
 Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to:
 

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

 Apply data hiding so salary components are not directly accessible.  
OOP Concepts: Encapsulation | Data Hiding | Methods

**Your Answer**

```

START
CLASS Employee
PRIVATE
    empId
    name
    basicPay
    designation
    grossSalary
    netSalary

PUBLIC:
Method acceptDetails()
    Read empId
    Read name
    Read basicPay
    Read designation
End Method

Method calculateGrossSalary()
    HRA = basicPay * 20 / 100
    DA = basicPay * 10 / 100
    grossSalary = basicPay + HRA + DA
End Method

Method calculateNetSalary()
    PF = basicPay * 5 / 100
    Tax = basicPay * 2 / 100
    netSalary = grossSalary - PF - Tax
          
```

4.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**PSEUDOCODE WRITING**
← Back

**QUESTIONS**

**Q1** Student Management System (BL3 - Apply) 10 marks

**Q2** Library Book Management (BL3 - Apply) 10 marks

**Q3** Employee Salary Calculation (BL3 - Apply) 10 marks

**Q4** Shape Area Calculator using Polymorphism (BL3 - Apply) 10 marks

**Q5** Vehicle Registration System (BL3 - Apply) 10 marks

**Q6** Bank Account Hierarchy (BL4 - Analyze) 10 marks

**Q7** Hospital Patient Record System (BL4 - Analyze) 10 marks

**Q8** E-commerce Product Catalog (BL4 - Analyze) 10 marks

**Q9** University Staff Hierarchy (BL4 - Analyze) 10 marks

**Q10**

**Q4 Shape Area Calculator using Polymorphism [BL3 - Apply]** 10 marks
 Create a Shape base class with an overridable calculateArea() method. Apply this to:
 

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

 Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.  
OOP Concepts: Inheritance | Polymorphism | Method Overriding

**Your Answer**

```

START
CLASS Shape
Method calculateArea()
    Print "Area of Shape"
End Method
END CLASS

CLASS Circle EXTENDS Shape
Method calculateArea()
    Read radius
    area = 3.14 * radius * radius
    Print area
End Method
END CLASS

CLASS Rectangle EXTENDS Shape
Method calculateArea()
    Read length
    Read breadth
    area = length * breadth
    Print area
End Method
END CLASS

CLASS Triangle EXTENDS Shape
Method calculateArea()
    Read base
    Read height
    area = (base * height) / 2
    Print area
End Method
          
```

5.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**SADHRI HARSHA VARDHAN**  
STUDENT

**PSEUDOCODE WRITING**

QUESTIONS

Q1 Student Management System (BL3 - Apply) 10 marks

Q2 Library Book Management (BL3 - Apply) 10 marks

Q3 Employees Salary Calculation (BL3 - Apply) 10 marks

Q4 Shape Area Calculator using Polymorphism (BL3 - Apply) 10 marks

Q5 5. Vehicle Registration System (BL3 - Apply) 10 marks

Q6 6. Bank Account Hierarchy (BL4 - Analyze) 10 marks

Q7 7. Hospital Patient Record System (BL4 - Analyze) 10 marks

Q8 8. E-Commerce Product Catalog (BL4 - Analyze) 10 marks

Q9 9. University Staff Hierarchy (BL4 - Analyze) 10 marks

Q10 10. Smart Home Device Controller (BL4 - Analyze) 10 marks

**5. Vehicle Registration System (BL3 - Apply)**
10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:
 

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

 Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.  
OOP Concepts: Inheritance | Method Overriding | Subclass

**Your Answer**

```

START
CLASS Vehicle
  regNo
  owner
  fuelType
  Method displayDetails()
    Print regNo
    Print owner
    Print fuelType
  End Method
END CLASS

CLASS TwoWheeler EXTENDS Vehicle
  engineCapacity
  Method displayDetails()
    Print regNo
    Print owner
    Print fuelType
    Print engineCapacity
  End Method
END CLASS

CLASS FourWheeler EXTENDS Vehicle
  seatingCapacity
  ACStatus
  Method displayDetails()
    Print regNo
    Print owner
    Print fuelType
    Print seatingCapacity
    Print ACStatus
          
```

6.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**SADHRI HARSHA VARDHAN**  
STUDENT

**PSEUDOCODE WRITING**

QUESTIONS

Q1 Student Management System (BL3 - Apply) 10 marks

Q2 Library Book Management (BL3 - Apply) 10 marks

Q3 Employees Salary Calculation (BL3 - Apply) 10 marks

Q4 Shape Area Calculator using Polymorphism (BL3 - Apply) 10 marks

Q5 5. Vehicle Registration System (BL3 - Apply) 10 marks

Q6 6. Bank Account Hierarchy (BL4 - Analyze) 10 marks

Q7 7. Hospital Patient Record System (BL4 - Analyze) 10 marks

Q8 8. E-Commerce Product Catalog (BL4 - Analyze) 10 marks

Q9 9. University Staff Hierarchy (BL4 - Analyze) 10 marks

Q10 10. Smart Home Device Controller (BL4 - Analyze) 10 marks

**6. Bank Account Hierarchy (BL4 - Analyze)**
10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:
 

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

 Compare the behavior of withdraw() across both subclasses and justify the design choices.  
OOP Concepts: Inheritance | Method Overriding | Polymorphism

**Your Answer**

```

START
CLASS BankAccount
  accNo
  holder
  balance
  Method withdraw(amount)
    IF amount <= balance
      balance = balance - amount
      Print "Amount Withdrawn"
    ELSE
      Print "Insufficient Balance"
    END IF
  End Method
END CLASS

CLASS SavingsAccount EXTENDS BankAccount
  Method withdraw(amount)
    IF amount <= balance
      balance = balance - amount
      Print "Withdrawal Successful"
    ELSE
      Print "Cannot Withdraw - Low Balance"
    END IF
  End Method
END CLASS

CLASS CurrentAccount EXTENDS BankAccount
  overdraftLimit
  Method withdraw(amount)
          
```

7.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**SADHRI HARSHA VARDHAN**  
12/03/2020

**PSEUDOCODE WRITING**

Q1

Student Management System (BL3 - Apply)

10 marks

Q2

Library Book Management (BL3 - Apply)

10 marks

Q3

Employee Salary Calculation (BL3 - Apply)

10 marks

Q4

Movie Area Calculator using Polymorphism (BL3 - Apply)

10 marks

Q5

Vehicle Registration System (BL3 - Apply)

10 marks

Q6

Bank Account Hierarchy (BL4 - Analyze)

10 marks

Q7

7. Hospital Patient Record System (BL4 - Analyze)

10 marks

Q8

8. E-Commerce Product Catalog (BL4 - Analyze)

10 marks

Q9

9. University Staff Hierarchy (BL4 - Analyze)

10 marks

Q10

Q7

7. Hospital Patient Record System (BL4 - Analyze)

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```

patientId
name
diagnosis
prescription
PUBLIC
Method generateBill()
Print "Patient Bill"
End Method
END CLASS
CLASS InPatient EXTENDS Patient
wardNumber
admissionDate
dailyCharges
days
Method generateBill()
bill = dailyCharges * days
Print bill
End Method
END CLASS
CLASS OutPatient EXTENDS Patient
appointmentDate
consultationFee
Method generateBill()
bill = consultationFee
Print bill
End Method
END CLASS
Create InPatient object
Call generateBill()
          
```

8.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**SADHRI HARSHA VARDHAN**  
12/03/2020

**PSEUDOCODE WRITING**

Q1

Student Management System (BL3 - Apply)

10 marks

Q2

Library Book Management (BL3 - Apply)

10 marks

Q3

Employee Salary Calculation (BL3 - Apply)

10 marks

Q4

Movie Area Calculator using Polymorphism (BL3 - Apply)

10 marks

Q5

Vehicle Registration System (BL3 - Apply)

10 marks

Q6

Bank Account Hierarchy (BL4 - Analyze)

10 marks

Q7

7. Hospital Patient Record System (BL4 - Analyze)

10 marks

Q8

8. E-Commerce Product Catalog (BL4 - Analyze)

10 marks

Q9

9. University Staff Hierarchy (BL4 - Analyze)

10 marks

Q10

Q8

8. E-Commerce Product Catalog (BL4 - Analyze)

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```

START
CLASS Product
productId
name
price
Method applyDiscount()
Print "Discount Applied"
End Method
END CLASS
CLASS Electronics EXTENDS Product
warranty
brand
voltage
Method applyDiscount()
discount = price * 10 / 100
finalPrice = price - discount
Print finalPrice
End Method
END CLASS
CLASS Clothing EXTENDS Product
size
fabric
gender
Method applyDiscount()
discount = price * 20 / 100
finalPrice = price - discount
Print finalPrice
End Method
END CLASS
CLASS Grocery EXTENDS Product
expiryDate
weight
Method applyDiscount()
discount = price * 5 / 100
finalPrice = price - discount
Print finalPrice
End Method
END CLASS
Create Electronics object
Call applyDiscount()
          
```

9.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**BAHINI EKADEKA VARDHAN**  
TECHNICAL

**PSEUDOCODE WRITING**

QUESTIONS

Q1 Student Management System (20 - Marks)

Q2 Library Book Management (20 - Marks)

Q3 Bank System Salary Calculation (20 - Marks)

Q4 Simple Bank Loan System using Abstraction (20 - Marks)

Q5 Vehicle Registration System (20 - Marks)

Q6 Bank Account Hierarchy (20 - Marks)

Q7 Hospital Patient Record System (20 - Marks)

Q8 E-commerce Product Listing (20 - Marks)

Q9 University Staff Hierarchy (20 - Marks)

Q10 Smart Home Device (20 - Marks)

10/10 answered

9. University Staff Hierarchy (20 - Marks)

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private or protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```

START
CLASS Staff
PRIVATE:
    name
PROTECTED:
    basicSalary
Method calculateAllowance()
    Print "Staff Allowance"
End Method
END CLASS
CLASS TeachingStaff EXTENDS Staff
subject
Method calculateAllowance()
    allowance = basicSalary * 20 / 100
    Print allowance
End Method
END CLASS
CLASS Professor EXTENDS TeachingStaff
experience
Method calculateAllowance()
    allowance = basicSalary * 30 / 100
    Print allowance
End Method
END CLASS
CLASS Lecturer EXTENDS TeachingStaff
qualification
Method calculateAllowance()
    allowance = basicSalary * 15 / 100
    Print allowance
End Method
END CLASS
CLASS NonTeachingStaff EXTENDS Staff
department
Method calculateAllowance()
    allowance = basicSalary * 10 / 100
    Print allowance
End Method

```

Save Answer

10.

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
JAVA PROGRAMMING

**BAHINI EKADEKA VARDHAN**  
TECHNICAL

**PSEUDOCODE WRITING**

QUESTIONS

Q1 Student Management System (20 - Marks)

Q2 Library Book Management (20 - Marks)

Q3 Bank System Salary Calculation (20 - Marks)

Q4 Simple Bank Loan System using Abstraction (20 - Marks)

Q5 Vehicle Registration System (20 - Marks)

Q6 Bank Account Hierarchy (20 - Marks)

Q7 Hospital Patient Record System (20 - Marks)

Q8 E-commerce Product Listing (20 - Marks)

Q9 University Staff Hierarchy (20 - Marks)

Q10 Smart Home Device (20 - Marks)

10/10 answered

10. Smart Home Device Controller (20 - Marks)

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand() method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed). Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```

START
CLASS SmartDevice
Method executeCommand(cmd)
    Print "Execute Command"
End Method
END CLASS
CLASS SmartLight EXTENDS SmartDevice
Method executeCommand(cmd)
    If cmd = "on"
        Print "Light ON"
    ELSE If cmd = "off"
        Print "Light OFF"
    ELSE
        Print "Light Status"
    END IF
End Method
END CLASS
CLASS SmartThermostat EXTENDS SmartDevice
Method executeCommand(cmd)
    If cmd = "on"
        Print "Thermostat ON"
    ELSE If cmd = "off"
        Print "Thermostat OFF"
    ELSE
        Print "Temperature Status"
    END IF
End Method
END CLASS
CLASS SmartLock EXTENDS SmartDevice
PRIVATE:
    PIN
Method executeCommand(cmd)
    If cmd = "on"
        Print "Lock Activated"
    ELSE If cmd = "off"
        Print "Lock Opened"
    ELSE
        Print "Lock Status"
    END IF
End Method

```

Save Answer